Phase 1 Modification Plan

Vyapar AI — Universal AI Employee Platform

Document: Phase 1 Implementation Plan (Approval Required)

Project Path: Z:\Vyapar AI\cursor

Date: June 1, 2026

Status: DRAFT — AWAITING APPROVAL

Goal: Single-tenant HONS MVP लाई multi-tenant-ready AI Employee foundation मा रूपान्तरण

1. Executive Summary (संक्षिpt)

यो Phase 1 plan ले `company_profiles.json` लाई **single source of truth** बनाउँछ। Hardcoded HONS data (`business_settings.py` , `products.py`) हटाइन्छ। प्रत्येक deployment `COMPANY_ID` environment variable बाट tenant identify गर्छ।

- Memory system — preserved
- Telegram webhook/polling — preserved
- OpenAI engine — preserved
- Admin panel, vector DB, WhatsApp/voice — **NOT in scope**

2. Scope Guardrails

Include	Exclude
company_profiles.json as SSOT	Admin panel
company_manager API expansion	Vector database / RAG
COMPANY_ID env routing	WhatsApp / voice channels
Error handling for missing profile	Package restructure (src/)
env.example update	DB tenant columns (Phase 2)

3. Files to Modify (7)

#	File	Action	Description
1	`company_profiles.json`	Modify	Schema enrich: products (tags, description), rules array
2	`company_manager.py`	Modify	New API functions + CompanyProfileError
3	`business_settings.py`	Modify	Load from company_manager; remove hardcoded constants
4	`products.py`	Modify	Dynamic product load per company_id
5	`ai_employee_engine.py`	Modify	Pass COMPANY_ID through reply pipeline
6	`env.example`	Modify	Add COMPANY_ID=hons
7	`main.py`	Modify (minimal)	Startup validation log for company profile

4. Files Unchanged (Preserved)

File	Reason
`memory.py`, `memory_db.py`, `memory_extractor.py`	Memory system preserved
`openai_engine.py`	OpenAI engine preserved
`main.py` webhook/polling/rate limit	Telegram preserved
`intent_engine.py`, `knowledge_base.py`, `prompts.py`	Working features preserved
`admin_*.py`, `render.yaml`, `requirements.txt`	Out of Phase 1 scope

5. company_profiles.json — Schema Enrichment

Problem: Current JSON lacks `tags`, `description`, and explicit `rules` needed for search quality and prompt rules.

Proposed structure (per tenant):

```
{
  "hons": {
    "company_name": "Himalayan Online Service Pvt. Ltd.",
    "business_type": "Internet Service Provider",
    "location": "Head Kamal Pokhari, Kathmandu, Nepal",
    "contact": {
      "phone": "+977-1-4541063",
      "toll_free": "16600149520",
      "email": "info@hons.com.np"
    },
    "support_hours": "6:00 AM to 7:00 PM",
    "installation_charge": "Free",
    "router": "Free",
    "onu": "Free",
    "rules": [
      "Always identify the company by its official name when needed.",
      "Do not invent package prices.",
      "If customer asks package price, use only the available product data.",
      "Installation is free.",
      "Router/ONU is free.",
      "Support is available during the listed support hours.",
      "For urgent support, provide phone or toll-free number."
    ],
    "products": [
      {
        "name": "100 Mbps Internet Package",
        "price": 10000,
        "duration_months": 13,
        "tags": ["100mbps", "100 mbps", "internet", "package", "isp"],
        "description": "100 Mbps internet package. Installation and Router/ONU are free."
      }
    ]
  }
}
```

Same pattern for 150 Mbps and 200 Mbps — **behavior parity** with current `products.py`.

6. company_manager.py — New API

```
class CompanyProfileError(Exception):
    """Raised when company profile is missing or invalid."""

def get_company(company_id: str) -> dict | None
def get_company_products(company_id: str) -> list[dict]
def get_company_contact(company_id: str) -> dict
def get_company_rules(company_id: str) -> list[str]
def get_company_summary(company_id: str) -> str
def require_company(company_id: str) -> dict
```

Function	Returns / Behavior
`get_company`	Raw tenant dict or None
`get_company_products`	Normalized: name, price ("NPR 10,000"), duration ("13 months"), tags, description
`get_company_contact`	{phone, toll_free, email} with {} fallbacks
`get_company_rules`	rules list from JSON; auto-derive if missing
`require_company`	Raises CompanyProfileError with clear message

7. business_settings.py — Refactor

Before: 12 hardcoded constants + static HONS prompt.

After:

```
def business_context_to_prompt(company_id: str) -> str:
    company = require_company(company_id)
    contact = get_company_contact(company_id)
    rules = get_company_rules(company_id)
    # Build prompt dynamically from company + contact + rules
```

- Remove all BUSINESS_NAME, BUSINESS_PHONE, etc. constants
- `company_id` is required parameter
- Prompt output structurally identical to today for HONS

8. products.py — Dynamic Load

```
def _load_products(company_id: str) -> list[dict]
def search_products(query, top_n=3, company_id: str) -> list[dict]
def products_to_prompt(items, company_id: str) -> str
```

- Remove module-level PRODUCTS constant
- Search algorithm unchanged (tags, name, price keywords)

- Fallback listing from tenant JSON, not hardcoded Mbps lines

9. ai_employee_engine.py — Tenant-Aware Replies

```
COMPANY_ID = os.getenv("COMPANY_ID", "hons").strip()

async def generate_employee_reply(user_id: str, text: str) -> str:
    company_id = COMPANY_ID
    try:
        require_company(company_id)
    except CompanyProfileError:
        return "This AI employee is not configured yet. Please contact support."

    product_items = search_products(clean_text, top_n=3, company_id=company_id)
    system_prompt = compose_system_prompt(
        business_block=business_context_to_prompt(company_id),
        product_block=products_to_prompt(product_items, company_id=company_id),
        ...
    )
```

Preserved: Memory extraction, direct recall, intent, knowledge base, OpenAI call, Telegram 3500 char cap.

10. env.example — Add Tenant Config

```
COMPANY_ID=hons
```

Stale unused vars (BUSINESS_NAME, SUPPORT_PHONE, etc.) comment/remove — loaded from company_profiles.json.

11. main.py — Startup Validation

```
# In lifespan, after init_db():
company_id = os.getenv("COMPANY_ID", "hons").strip()
try:
    require_company(company_id)
    logger.info("COMPANY_PROFILE_LOADED company_id=%s", company_id)
except CompanyProfileError:
    logger.error("COMPANY_PROFILE_MISSING company_id=%s", company_id)
```

App still starts (Telegram/health work); misconfiguration visible in logs immediately.

12. Data Flow (After Phase 1)

```
COMPANY_ID (env)
    ↓
ai_employee_engine.py
    ↓
company_manager.py ← company_profiles.json (SSOT)
    ↓                    ↓
business_settings.py   products.py
    ↓
prompts.py → openai_engine.py → Telegram reply

memory.py (unchanged, parallel path)
```

13. Error Handling Matrix

Scenario	Behavior
COMPANY_ID missing in env	Default "hons"
company_profiles.json missing	CompanyProfileError → user-friendly reply + error log
Unknown COMPANY_ID	Same as above
Company exists but no products	Empty search; prompt says no packages confirmed
Company exists but no rules	Auto-derived from install/router/onu/support_hours

14. Manual Testing Plan

1. `COMPANY_ID=hons` → same HONS package/pricing answers as before
2. `COMPANY_ID=invalid` → graceful error message, no crash
3. Memory: "Mero naam X ho" / "Mero naam ke ho?" still works
4. Telegram webhook + OpenAI path unchanged
5. Invalid/missing JSON → startup error log visible

15. Risks & Non-Goals

Item	Decision
Multi-bot single process	One COMPANY_ID per deployment (Render instance)
Hot reload JSON	Not included — restart required after profile change
company_profile.json	Not used; optional delete later
DB tenant column	Phase 2 — not in this PR

16. Estimated Diff Size

~250–350 lines changed across 7 files. No new directories.

17. Approval Checklist

> **Please confirm before implementation:**
>
> 1. Approve all 7 file changes as listed?
> 2. Enrich company_profiles.json with rules, product tags/description?
> 3. Default COMPANY_ID=hons when env unset?
> 4. Delete company_profile.json or leave unused?
> 5. Update README_DEPLOY.md with COMPANY_ID (recommended)?
>
> Reply **"approved"** with any adjustments to proceed with implementation.
